RAG

↳ Retrieval - Augmented Generation.
(fetching).

⇒ Why RAG is needed ?

LLMs are powerful but they have some limitations :

→ Knowledge can be outdated.
→ No access to private data by default.
→ Hallucination risk.

RAG solves these limitations by :
→ Connecting LLM to external knowledge source.

Retrieval ⇒ finding the information from relavant sources.

Generation ⇒ Writing / framing the answers using this information.

RAG Pipeline flow :

→ Documents Uploading.

→ Document Parsing.

→ Chunking.

→ Metadata Extraction.

→ Embedding Creation

→ Store embeddings in Vector DB.

→ Query processing

- → Retrieve information from Vector DB.
- → Generate the answers.

RAG is extremely powerful wherever answer must come from trusted / internal info. sources.

RAG Use-Cases.

→ Customer Support Assistants.

→ Legal AI Assistants

→ Education / Learning Assistants

Document Uploading & Parsing

→ Before the retrieval | generations, the documents must be brought into the pipeline.

→ Document loading means collecting documents from various sources & uploading them.

Document Parsing

→ Extracting meaningful | useful information from documents.

→ PDF parser | doc parser | . . . .

→ Parsing reduces unwanted | repeated information from the documents.

Chunking

→ Splitting the document into smaller pieces

→ Chunking helps in managing the documents easily & it makes the retrieval faster.

Chunking Strategies.

→ Fixed Chunking Strategy.

- Words | pages | - - -
- Simplest approach.
- Easy to implement.
- May cut a sentence into multiple chunks.
  - → Loss of information.
- Retrieval quality may suffer.

→ Semantic Chunking Strategy.

- Chunks are created based on the meaning rather than fixed length.
- More natural.
- better retrieval quality.
- preserves the context better.
- Processing can be very slow & expensive.

→ Recursive Chunking Strategy.

- More practical & frequently used.
- Splits the text using hierarchy of separators.
  - headings / topics.
  - Paragraphs
  - Sentences.

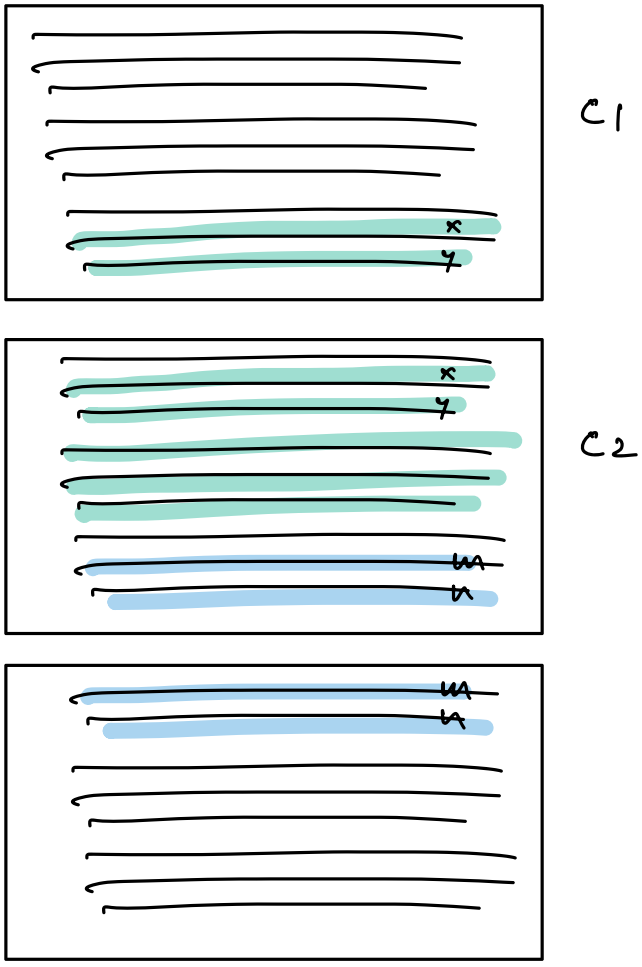
- More practical
- Preserves document structure.
- Widely used in RAG.

- May not be perfect chunks.
- Some chunks can be very big &
  Some chunks can be very small.

Chunk Size Optimization.

Chunk Overlap.

↳ Repeating a small portion of text between consecutive chunks.



Document Metadata Extraction.

↳ data about data.

⇒ Maintain some extra information about the data to make retrieval faster & efficient.